Lab 4: Enhanced Grade Calculator with Input Validation

Learning Objectives

By completing this lab, students will be able to:

- Apply the IPO model to program design.
- Create pseudocode and flowcharts.
- Use relational operators.
- Use if, if-else, and if-else if statements.
- Validate user input.
- Identify invalid data.
- Explain how future topics such as loops can improve program design.

Background

In previous assignments, your program calculated the average of three test scores and displayed a letter grade. What if the user enters invalid test scores (e.g., -15, 125, etc.)? Real-world software must be able to identify and handle invalid input.

In this lab, you will improve your existing program by validating user input before calculating the average and letter grade.

Problem Statement

Create a program that asks the user to select the course for which they want to calculate the average. **This must be a switch statement to display the menu and evaluate the user's input.** Then, ask the user to enter 5 test scores. Check whether any of the scores are less than 0 or greater than 100. If any of the scores are invalid, your program should display an error message and not calculate the average. If all scores are valid:

- Calculate the average.
- Determine the letter grade.
- Display the average and letter grade.

Here are the available courses:

- CSC 221 - Fundamental Programming I
- MAT 131 - College Algebra
- BIO 111 - General Biology

The grading scale is as follows:

- 90 – 100 = A

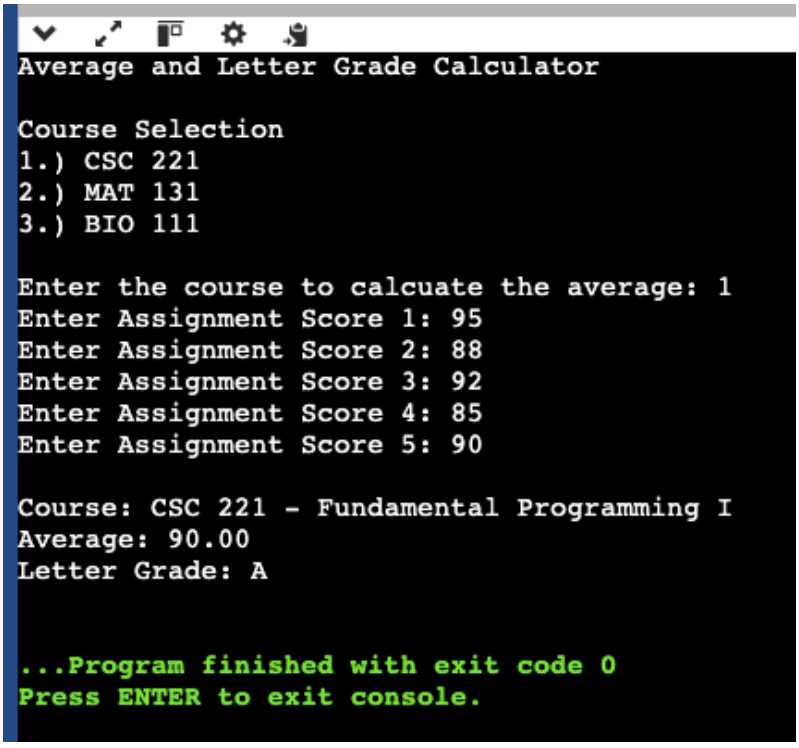
- 80 – 89 = B
- 70 – 79 = C
- 60 – 69 = D
- Below 60 = F

Design Requirement

You must create a design for this program. Design BEFORE coding! Write step-by-step instructions in English for your pseudocode. Use mathematical formulas as needed. Add blank lines and bullet points to make your design easy to read. You must also create a corresponding flowchart for this program.

Example Program Run

Your program should mirror the screenshot below. The user interface should include the exact same questions and report the exact same information. Only the user input and corresponding output may differ.



Implementation Requirements

Create a C++ program that includes the following:

- Meaningful variable names
- Comments

- Proper whitespace
- Input statements
- Output statements

Students will not be graded on advanced C++ syntax. Focus on problem-solving and program organization.

Testing

Test Case	Course Choice	Score 1	Score 2	Score 3	Score 4	Score 5	Course	Average	Letter Grade	Test Passed? (Y/N)
1	2	95	88	92	85	90	MAT 131 - College Algebra	90	A	
2	1	75	70	80	72	78				
3	3	85	90	105	88	92				

Lab Report

Your report must contain the sections listed below. Be sure to include your name in the document. Each section must have a clearly indicated header.

- Problem Statement
    - In a paragraph, describe the problem of your program and what it is meant to solve.
- Design
    - This section should include your pseudocode and flowchart
- Test Cases and Test Results
    - Submit the completed table in the Testing section.
- Screenshot
    - Submit a screenshot of your program using one of the three test cases in the Testing section.
- Analysis and Conclusions
    - Answer the following questions in complete sentences. There are no “right” or “wrong” answers.
        - What was the most difficult part of this assignment and how did you overcome the challenge(s)?
        - How does this lab demonstrate the IPO model?
        - Why is planning/design important before coding?

- Think outside of the program specifications. How could this program be improved?

Deliverables

1. C++ code file (must be a C++ file, not text, not Word, etc.)
    a. Example file name: Lab4_Program_LastName_FirstInitial.cpp
2. Lab report file (must be a PDF file, not text, not Word, etc.)
    a. Example file name: Lab4_Report_LastName_FirstInitial.pdf